在分布式缓存系统中，**一致性哈希（Consistent Hashing）** 是一种常用的**负载均衡策略**，用于解决缓存节点的动态扩展和缩容问题。它可以**减少缓存失效率，提高缓存命中率**，从而提高系统的可扩展性和稳定性。

为什么需要一致性哈希？

在分布式缓存系统中，多个服务器（缓存节点）存储不同的缓存数据，**客户端需要决定将某个 key 存储在哪个缓存节点**。最简单的方式是使用 **取模（Modulo）分片**：

对于分布式缓存来说，当一个节点接收到请求，如果该节点并没有存储缓存值，那么它面临的难题是，从谁那获取数据？自己，还是节点1, 2, 3, 4⋯ 。假设包括自己在内一共有 10 个节点，当一个节点接收到请求时，随机选择一个节点，由该节点从数据源获取数据。

假设第一次随机选取了节点 1 ，节点 1 从数据源获取到数据的同时缓存该数据；那第二次，只有 1/10 的可能性再次选择节点 1, 有 9/10 的概率选择了其他节点，如果选择了其他节点，就意味着需要再一次从数据源获取数据，一般来说，这个操作是很耗时的。这样做，一是缓存效率低，二是各个节点上存储着相同的数据，浪费了大量的存储空间。

那有什么办法，对于给定的 key，每一次都选择同一个节点呢？使用 hash 算法能够做到这一点。那把 key 的每一个字符的 ASCII 码加起来，再除以 10 取余数可以吗？当然可以，这可以认为是自定义的 hash 算法。

代码块

```go
// 传统哈希分片示例
func getShard(key string, nodeCount int) int {
    hash := crc32.ChecksumIEEE([]byte(key))
    return int(hash) % nodeCount
}
```

节点数量变化了怎么办？

但这种方式存在以下问题：

- **当节点数量变更时，大量 key 的缓存映射会发生变化**，导致缓存命中率大幅下降。
- **数据迁移成本高**，每次增加或删除节点，都需要重新计算所有 key 的存储位置。

简单求取 Hash 值解决了缓存性能的问题，但是没有考虑节点数量变化的场景。假设，移除了其中一台节点，只剩下 9 个，那么之前 hash(key) % 10 变成了 hash(key) % 9，也就意味着几乎缓存值对应的节点都发生了改变。即几乎所有的缓存值都失效了。节点在接收到对应的请求时，均需要重新去数据源获取数据，容易引起 缓存雪崩。

那如何解决这个问题呢？一致性哈希算法可以。

一致性哈希算法 通过构建哈希环，使得节点变更时仅影响部分缓存数据，从而减少缓存失效，提高系统的可用性。

算法原理

一致性哈希使用一个**虚拟的哈希环**（0~2^{32}），所有缓存节点和 key 通过 **哈希函数**（如 `MurmurHash`、 `FNV-1a`）映射到环上的某个位置。

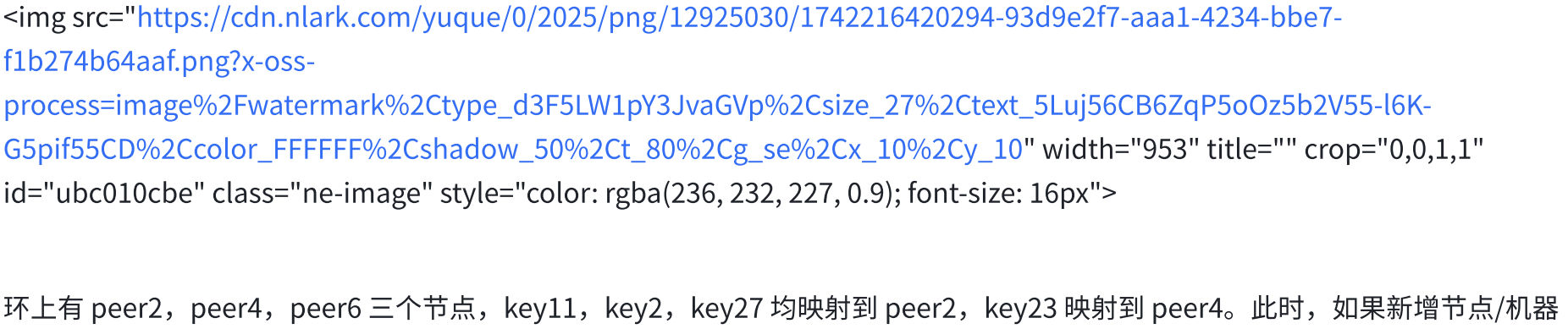
- **节点映射**：将缓存节点的 IP 或名称进行哈希计算，并映射到哈希环上。
- **数据映射**：将 key 计算哈希值，并找到**顺时针方向最近的节点**，作为 key 的存储位置。

步骤

一致性哈希算法将 key 映射到 2^32 的空间中，将这个数字首尾相连，形成一个环。

● 计算节点/机器(通常使用节点的名称、编号和 IP 地址)的哈希值，放置在环上。

● 计算 key 的哈希值，放置在环上，顺时针寻找到的第一个节点，就是应选取的节点/机器。



环上有 peer2，peer4，peer6 三个节点，key11，key2，key27 均映射到 peer2，key23 映射到 peer4。此时，如果新增节点/机器 peer8，假设它新增位置如图所示，那么只有 key27 从 peer2 调整到 peer8，其余的映射均没有发生改变。

也就是说，一致性哈希算法，在新增/删除节点时，只需要重新定位该节点附近的一小部分数据，而不需要重新定位所有的节点，这就解决了上述的问题。

数据倾斜问题

如果服务器的节点过少，容易引起 key 的倾斜。例如上面例子中的 peer2，peer4，peer6 分布在环的上半部分，下半部分是空的。那么映射到环下半部分的 key 都会被分配给 peer2，key 过度向 peer2 倾斜，缓存节点间负载不均。

为了解决这个问题，引入了虚拟节点的概念，一个真实节点对应多个虚拟节点。

假设 1 个真实节点对应 3 个虚拟节点，那么 peer1 对应的虚拟节点是 peer1-1、 peer1-2、 peer1-3（通常以添加编号的方式实现），其余节点也以相同的方式操作。

● 第一步，计算虚拟节点的 Hash 值，放置在环上。

● 第二步，计算 key 的 Hash 值，在环上顺时针寻找到应选取的虚拟节点，例如是 peer2-1，那么就对应真实节点 peer2。

虚拟节点扩充了节点的数量，解决了节点较少的情况下数据容易倾斜的问题。而且代价非常小，只需要增加一个字典(map)维护真实节点与虚拟节点的映射关系即可。

● 问题：如果节点分布不均匀，某些节点可能承载过多数据（数据倾斜）。

● 解决方案：使用虚拟节点，即每个实际节点在哈希环上映射多个位置，平衡数据分布。

这样，即使部分物理节点故障，数据仍能分散到多个虚拟节点，提高系统的均衡性和容错能力。

KamaCache 实现

核心数据结构

代码块

```
type Map struct {
    mu sync.RWMutex           // 读写锁，保证并发安全
    config *Config            // 配置信息
    keys []int                // 哈希环上的所有虚拟节点位置，按顺序排列
    hashMap map[int]string    // 从哈希值到实际节点名称的映射
    nodeReplicas map[string]int // 每个实际节点对应的虚拟节点数量
    nodeCounts map[string]int64 // 记录每个节点处理的请求数
    totalRequests int64       // 记录总请求数，用于负载均衡计算
}
```

- `mu`：读写互斥锁，用于保护并发访问。读写锁相比普通互斥锁的优势是允许多个读操作并发执行，只有写操作需要独占锁。
- `config`：存储配置信息，包含哈希函数、默认虚拟节点数等。
- `keys`：有序整数数组，存储所有虚拟节点的哈希位置，按升序排列以便二分查找。
- `hashMap`：将哈希环上的位置映射到实际节点名称，当查找到一个位置时，通过这个映射找到对应的实际节点。
- `nodeReplicas`：记录每个实际节点当前拥有的虚拟节点数量，用于负载均衡调整。
- `nodeCounts`：统计每个节点处理的请求数量，作为负载均衡的依据。
- `totalRequests`：记录总请求数，用于计算平均负载。

节点管理

代码块

```
func (m *Map) Add(nodes ...string) error {
    if len(nodes) == 0 {
        return errors.New("no nodes provided")
    }

    m.mu.Lock()
    defer m.mu.Unlock()

    for _, node := range nodes {
        if node == "" {
            continue
        }

```

```go
        // 为节点添加虚拟节点
        m.addNode(node, m.config.DefaultReplicas)
    }

    // 重新排序
    sort.Ints(m.keys)
    return nil
}

func (m *Map) addNode(node string, replicas int) {
    for i := 0; i < replicas; i++ {
        hash := int(m.config.HashFunc([]byte(fmt.Sprintf("%s-%d", node, i))))
        m.keys = append(m.keys, hash)
        m.hashMap[hash] = node
    }
    m.nodeReplicas[node] = replicas
}

func (m *Map) Remove(node string) error {
    if node == "" {
        return errors.New("invalid node")
    }

    m.mu.Lock()
    defer m.mu.Unlock()

    replicas := m.nodeReplicas[node]
    if replicas == 0 {
        return fmt.Errorf("node %s not found", node)
    }

    // 移除节点的所有虚拟节点
    for i := 0; i < replicas; i++ {
        hash := int(m.config.HashFunc([]byte(fmt.Sprintf("%s-%d", node, i))))
        delete(m.hashMap, hash)
        for j := 0; j < len(m.keys); j++ {
            if m.keys[j] == hash {
                m.keys = append(m.keys[:j], m.keys[j+1:]...)
                break
            }
        }
    }

    delete(m.nodeReplicas, node)
    delete(m.nodeCounts, node)
    return nil
}
```

请求路由

代码块

```go
func (m *Map) Get(key string) string {
    if key == "" {
        return ""
    }

    m.mu.RLock()
    defer m.mu.RUnlock()
```

```go

    if len(m.keys) == 0 {
        return ""
    }

    hash := int(m.config.HashFunc([]byte(key)))
    // 二分查找
    idx := sort.Search(len(m.keys), func(i int) bool {
        return m.keys[i] >= hash
    })

    // 处理边界情况
    if idx == len(m.keys) {
        idx = 0
    }

    node := m.hashMap[m.keys[idx]]
    count := m.nodeCounts[node]
    m.nodeCounts[node] = count + 1
    atomic.AddInt64(&m.totalRequests, 1)

    return node
}
```

路由查找的详细过程：

1. 参数校验：确保键不为空
2. 加读锁：允许多个查找操作并发进行
3. 检查哈希环是否为空
4. 使用相同的哈希函数计算键的哈希值
5. 使用标准库的 `sort.Search` 进行二分查找，寻找第一个大于或等于该哈希值的位置
6. 处理特殊情况：如果没有找到大于或等于的位置，表示应该环绕到哈希环的起点
7. 通过哈希映射获取对应的实际节点名称
8. 更新请求统计计数（注意这里没有使用额外的锁，因为读锁允许修改不影响并发安全的局部变量）
9. 使用原子操作增加总请求计数
10. 释放锁并返回节点名称

二分查找保证了查找操作的时间复杂度为O(log n)，其中n是虚拟节点的总数。

负载均衡机制

代码块

```go
func (m *Map) checkAndRebalance() {
    if atomic.LoadInt64(&m.totalRequests) < 1000 {
        return // 样本太少，不进行调整
    }

    // 计算负载情况
    avgLoad := float64(m.totalRequests) / float64(len(m.nodeReplicas))
    var maxDiff float64

```

```go
    for _, count := range m.nodeCounts {
        diff := math.Abs(float64(count) - avgLoad)
        if diff/avgLoad > maxDiff {
            maxDiff = diff / avgLoad
        }
    }

    // 如果负载不均衡度超过阈值，调整虚拟节点
    if maxDiff > m.config.LoadBalanceThreshold {
        m.rebalanceNodes()
    }
}

func (m *Map) rebalanceNodes() {
    m.mu.Lock()
    defer m.mu.Unlock()

    avgLoad := float64(m.totalRequests) / float64(len(m.nodeReplicas))

    // 调整每个节点的虚拟节点数量
    for node, count := range m.nodeCounts {
        currentReplicas := m.nodeReplicas[node]
        loadRatio := float64(count) / avgLoad

        var newReplicas int
        if loadRatio > 1 {
            // 负载过高，减少虚拟节点
            newReplicas = int(float64(currentReplicas) / loadRatio)
        } else {
            // 负载过低，增加虚拟节点
            newReplicas = int(float64(currentReplicas) * (2 - loadRatio))
        }

        // 确保在限制范围内
        if newReplicas < m.config.MinReplicas {
            newReplicas = m.config.MinReplicas
        }
        if newReplicas > m.config.MaxReplicas {
            newReplicas = m.config.MaxReplicas
        }

        if newReplicas != currentReplicas {
            // 重新添加节点的虚拟节点
            if err := m.Remove(node); err != nil {
                continue // 如果移除失败，跳过这个节点
            }
            m.addNode(node, newReplicas)
        }
    }

    // 重置计数器
    for node := range m.nodeCounts {
        m.nodeCounts[node] = 0
    }
    atomic.StoreInt64(&m.totalRequests, 0)

    // 重新排序
    sort.Ints(m.keys)
}
```

负载均衡检查的详细流程：

1. 首先检查总请求数是否达到有效样本（这里是1000），避免样本太少导致调整不准确
2. 计算平均负载：总请求数除以节点数
3. 计算最大负载偏差率：找出偏离平均负载最远的节点，计算其偏差占平均负载的比例
4. 如果最大偏差超过配置的阈值（例如20%），触发重新平衡操作

这种方法确保只有在负载确实不均衡时才进行调整，避免不必要的资源消耗。

重新平衡的详细算法：

1. 加写锁：确保在重新平衡期间没有其他操作
2. 重新计算平均负载（可能在获取锁期间有新请求）
3. 遍历每个节点，计算其负载比率（相对于平均负载）
4. 根据负载比率调整虚拟节点数量：
   - 对于负载过高的节点（比率>1），按比例减少虚拟节点
   - 对于负载过低的节点（比率<1），增加虚拟节点，增加幅度为(2-loadRatio)倍
5. 确保新的虚拟节点数量在配置的最小值和最大值范围内
6. 如果需要调整，先移除该节点所有虚拟节点，再用新的数量重新添加
7. 重置所有计数器，准备下一轮统计
8. 重新排序哈希环
9. 释放锁

这个算法的核心思想是：通过调整虚拟节点数量来影响后续请求的分发比例。负载高的节点减少虚拟节点，从而减少分配到该节点的请求；负载低的节点增加虚拟节点，吸引更多请求。

一致性哈希是分布式算法吗？

严格来说，一致性哈希（Consistent Hashing）本身并不是一个分布式算法，而是一种用于分布式系统的数据分布策略。

它的主要作用是在分布式存储或缓存系统中，将数据均匀地映射到不同的节点上，从而减少节点增删时的数据迁移，提高系统的可扩展性和稳定性。

虽然一致性哈希本身并不涉及节点间的通信和协调，但它是许多分布式算法的基础，比如：

- 分布式缓存（如 Groupcache、Memcached）
- 分布式存储（如 Amazon DynamoDB、Cassandra）
- 负载均衡（如 Nginx、Consul）

在分布式系统中，一致性哈希常用于 无中心化（decentralized）的架构，如 P2P 网络或去中心化存储。

<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);"></font>

<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);"></font>

<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);"></font>

与 Redis 的不同之处

对比项	KamaCache	Redis
架构类型	**P2P（无中心化）分布式缓存**	**基于主从的集中式分布式缓存**
数据存储	<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">每个实例都有部分缓存，数据由一致性哈希决定存放在哪个实例</font>	<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">数据存储在独立的 Redis 服务器或 Redis Cluster</font>
节点间通信	**节点互相发现**<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">，通过 Peer-to-Peer 直接访问</font>	**客户端-服务器模式**<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">，需要专门的 Redis 服务器</font>
数据一致性	<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">只缓存数据，依赖于底层数据库保证数据一致性</font>	<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);">通过</font>**主从复制（Replication）**<font style="color:rgba(236, 232, 227, 0.9);background-color:rgb(25, 26, 27);"> 保证数据同步</font>

一致性哈希的作用

● Groupcache 使用一致性哈希来决定 数据存放在哪个缓存节点，即 key 被映射到哪个 Groupcache 实例上。

● Redis（单实例模式） 直接存储数据，不涉及一致性哈希，而是让所有请求访问同一个 Redis 服务器。

● Redis Cluster 使用 哈希槽（Hash Slot）机制，通过 CRC16(key) % 16384 计算哈希槽，将 key 分片存储在不同的 Redis 节点上，实现数据均衡分布。

适用于哪些场景?

KamaCache 是一个分布式缓存，适用于：

- 无中心化的分布式架构（如微服务、P2P 系统）。
- 高性能读缓存，尤其适用于 防止缓存击穿。
- 避免集中式缓存的瓶颈，如单点 Redis 服务器的性能限制。